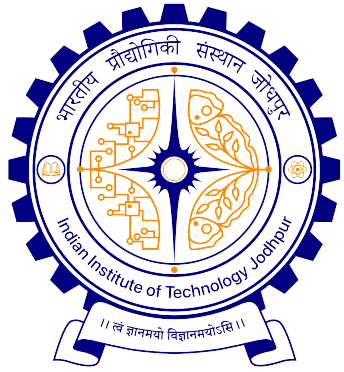


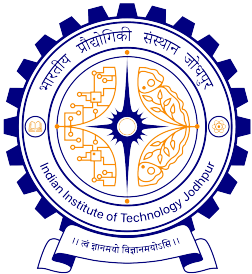


# Introduction to Computer Science (CSL1010)

## *C Basics– Arrays L8*

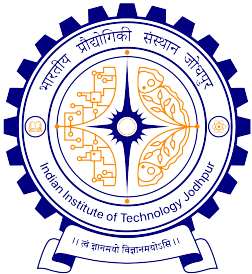
---

Suchetana Chakraborty  
Department of Computer Science, IIT Jodhpur  
Spring, 2026



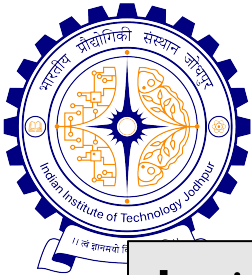
# Array

- For any primitive data type, one variable (container) can hold only one value at a time
  - Example: `int x = 100` – can only hold one integer value (100) at a time
- Imagine that you want to hold the names of 100 people at a time.
  - What would you have to do?
    - Declare 100 variables, one for each name.
  - Better solution: Use Arrays.
    - Arrays are complex variables that can hold multiple values of the same data type
    - Now we can declare a single array that holds all the names.



# Array

- Many applications require multiple data items that have common characteristics
  - In mathematics, we often express such groups of data items in indexed form (representing the relative order of these elements)
    - $x_1, x_2, x_3, \dots, x_n$
- Array is a data structure which can represent a collection of data items which have the same data type (float/int/char/...)

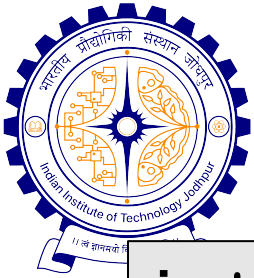


# Array Example

```
int main()
{
    int i;
    int data[10];
    for (i=0; i<10; i++) data[i]= i;
    i=0;
    while (i<10){
        printf("Data[%d] = %d\n", i, data[i]);
        i++;
    }
    return 0;
}
```

“data refers to a block of 10  
integer variables, data[0], data[1], ..., data[9]





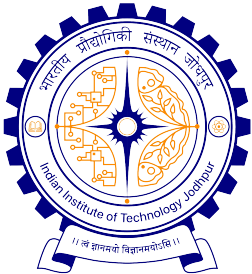
# Array Example

```
int main()  
{  
    int i;  
    int data[10];  
    for (i=0; i<10; i++) data[i]= i;  
    i=0;  
    while (i<10){  
        printf("Data[%d] = %d\n", i, data[i]);  
        i++;  
    }  
    return 0;  
}
```

Array size should be a constant

## Output

```
Data[0] = 0  
Data[1] = 1  
Data[2] = 2  
Data[3] = 3  
Data[4] = 4  
Data[5] = 5  
Data[6] = 6  
Data[7] = 7  
Data[8] = 8  
Data[9] = 9
```



# How is an array stored?

Starting from a given memory location, the successive array elements are allocated space in consecutive memory locations.

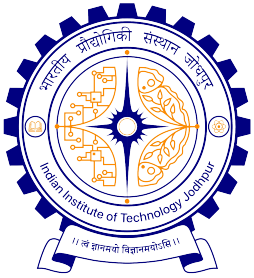
Array **a**



- **x**: starting address of the array in memory
- **k**: number of bytes allocated per array element

**a[i]** is allocated memory location at address

$$x + i * k$$

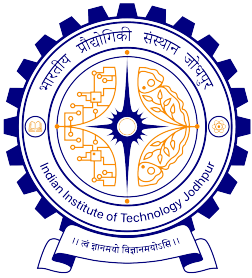


# Array Storage

```
int main()  
{  
    int i;  
    int data[10];  
    for(i=0; i<10; i++)  
        printf("&Data[%d] = %u\n", i, &data[i]);  
    return 0;  
}
```

## Output

```
&Data[0] = 3221224480  
&Data[1] = 3221224484  
&Data[2] = 3221224488  
&Data[3] = 3221224492  
&Data[4] = 3221224496  
&Data[5] = 3221224500  
&Data[6] = 3221224504  
&Data[7] = 3221224508  
&Data[8] = 3221224512  
&Data[9] = 3221224516
```



# Declaration

General form:

**type array-name [size];**

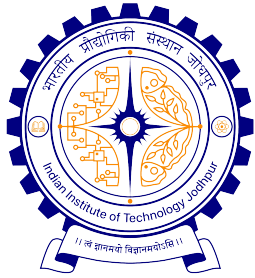
- **type** specifies the type of element that will be contained in the array (int, float, char, etc.)
- **size** is an integer constant which indicates the maximum number of elements that can be stored inside the array

**int marks[5];**

**marks** is an array that can store a maximum of 5 integers

**int D[10][20]**

A *one-dimensional array* with 10 elements, each of which is an array with 20 elements i.e., **int D[10][20] /\* [row] [col] \*/**



# Initializing at the time of declaration

General form:

```
type array_name[size] = { list of values };
```

Examples:

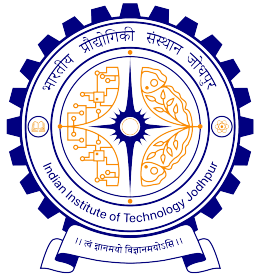
```
int marks[5] = {72, 83, 65, 80, 76};
```

```
char name[4] = {'A', 'm', 'i', 't'};
```

The size may be omitted. In such cases the compiler automatically allocates enough space for all initialized elements

```
int flag[ ] = {1, 1, 1, 0};
```

```
char name[ ] = {'A', 'm', 'i', 't'};
```



# More on Declaration

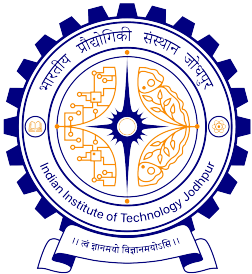
- Declaration can be **static** or **automatic**
- Array size may be determined explicitly or implicitly
- Outside of any function – always static

```
int A[13];
```

```
#define CLASS_SIZE 73  
double B[CLASS_SIZE];
```

```
const int nElements = 25  
float C[nElements];
```

```
static char S[256];    /*not visible to linker */
```



# More on Declaration

- Declaration can be **static** or **automatic**
- Array size may be determined explicitly or implicitly
- Outside of any function – always static

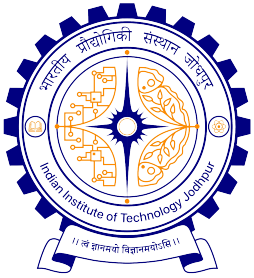
```
int A[13];
```

```
#define CLASS_SIZE 73  
double B[CLASS_SIZE];
```

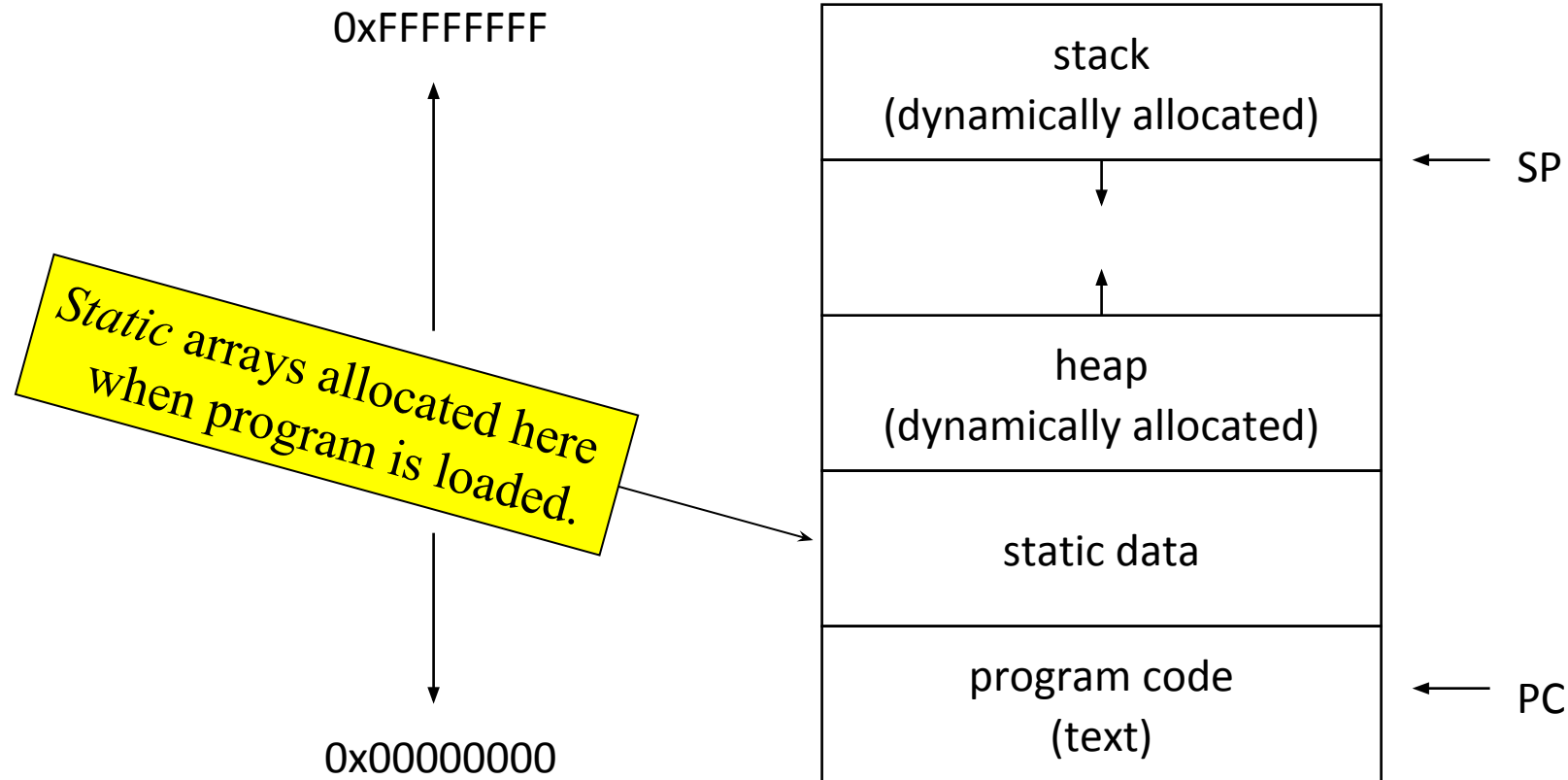
```
const int nElements = 25  
float C[nElements];
```

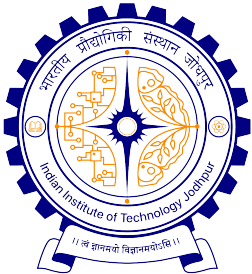
```
static char S[256];    /*not visible to linker */
```

*Static*  $\Rightarrow$  retains values across function calls



# Static Data Allocation



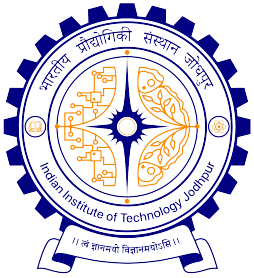


# Automatic Declaration

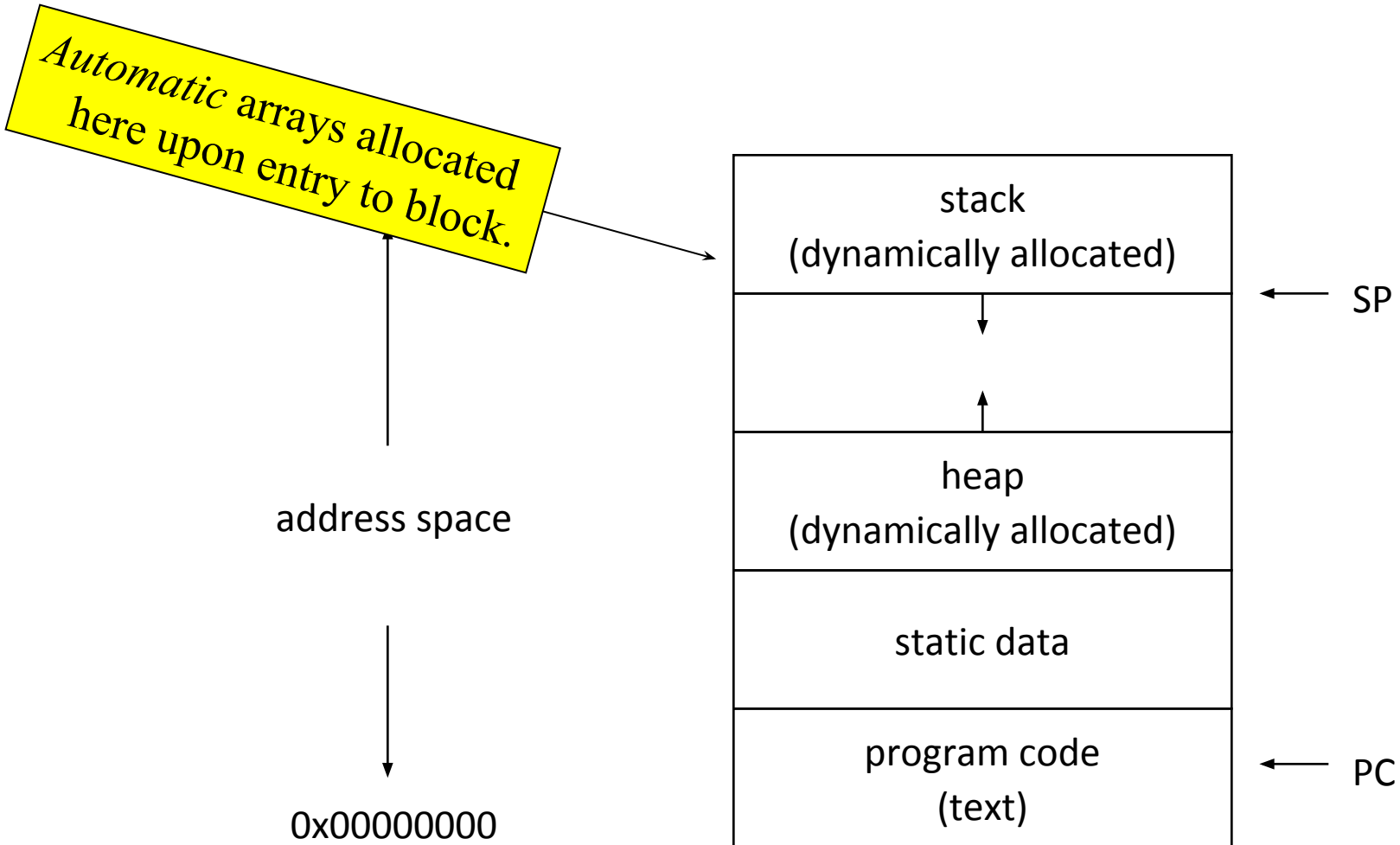
- Inside function or compound statement – usually automatic

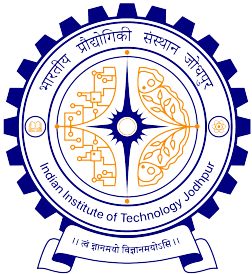
```
void f( ...) {  
    int A[13];  
  
    #define CLASS_SIZE 73  
    double B[CLASS_SIZE];  
  
    const int nElements = 25  
    float C[nElements];  
  
    static char D[256]; /*static, not visible outside function */  
  
} //f
```

This array is allocated in static data area when program is loaded



# Automatic Data Allocation





# Size of an array

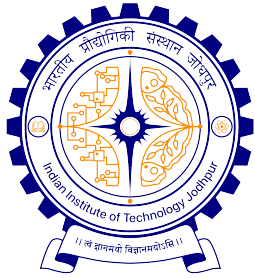
How do you keep track of the number of elements in the array ?

1. Use an integer to store the current size of the array.

```
#define MAX 100  
int size;  
float cost[MAX] ;
```

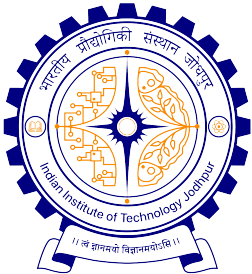
2. Use a special value to mark the last element in an array. If 10 values are stored, keep the values in `cost[0], ... , cost[9]`, have `cost[10] = -1`

3. Use the 0th array element to store the size (`cost[0]`), and store the values in `cost[1], ... , cost[cost[0]]`



# Adding an element to an array becomes easy

1. `cost[size] = newval; size++;`
2. `for (i=0; cost[i] != -1; i++) ;`  
`cost[i] = newval;`  
`cost[i+1] = -1;`
3. `cost[0]++;`  
`cost[cost[0]] = newval;`

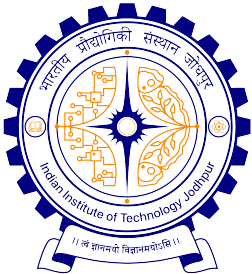


# Dynamic Array Size Determination

- **gcc** supports the following:—

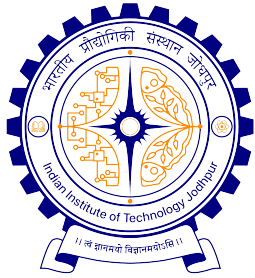
```
void func(<other parameters>, const int n) {  
    double Arr[2*n];  
  
} //func
```

- i.e., **array size is determined by evaluating an expression at run-time**
  - Automatic allocation on *The Stack*
  - Not in C88 ANSI standard, not in Kernighan & Ritchie
  - Part of C99 and C++



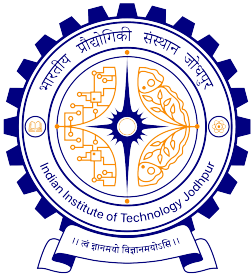
# Note on Array Initialization

- **`int A[5] = {2, 4, 8, 16, 32};`**
  - Static or automatic
- **`int B[20] = {2, 4, 8, 16, 32};`**
  - Unspecified elements are guaranteed to be zero
- **`int C[4] = {2, 4, 8, 16, 32};`**
  - Error — compiler detects too many initial values
- **`int D[5] = {2*n, 4*n, 8*n, 16*n, 32*n};`**
  - Automatically only; array initialized to expressions
- **`int E[n] = {1};`**
  - **gcc**, C99, C++
  - Dynamically allocated array (automatic only). Zeroth element initialized to *1*; all other elements initialized to *0*



# Implicit Array Size Determination

- `int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};`
  - Array is created with as many elements as initial values
    - In this case, 12 elements
  - Values must be compile-time constants (for static arrays)
  - Values may be run-time expressions (for automatic arrays)



# Getting Size of Implicit Array

- **sizeof** operator – returns # of bytes of memory required by operand

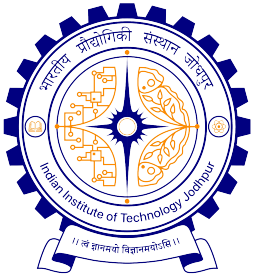
- Examples:–

- **sizeof (int)** – # of bytes per **int**
- **sizeof (float)** – # of bytes
- **sizeof days** – # of bytes in **days** (previous slide)
- # of elements in **days** =  $(\text{sizeof days}) / \text{sizeof(int)}$

**sizeof** with parentheses  
is size of the type

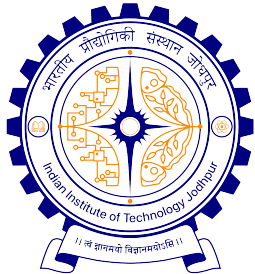
**sizeof** – no parentheses  
means size of the object

- Must be able to be determined at compile time
  - Getting size of dynamically allocated arrays not supported



# Accessing array elements

- A particular element of the array can be accessed by specifying two things:
  - **Name of the array**
  - **Index** (relative position) of the element in the array
- In C, the index of an array starts from **zero**
- Example:
  - An array is defined as **int x[10];**
  - The first element of the array x can be accessed as x[0], fourth element as x[3], tenth element as x[9], etc.



# Accessing array elements

- The array index must evaluate to an integer between 0 and  $n-1$  where  $n$  is the maximum number of elements possible in the array

```
a[x+2] = 25;
```

```
b[3*x-y] = a[10-x] + 5;
```

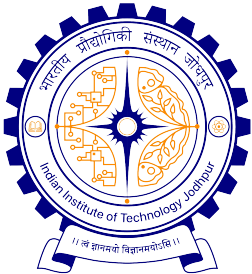
- Array elements are commonly used in loops

```
for(i=0; i < max; i++)
```

```
    A[i] = i*i;
```

```
for(sum = 0, j=0; j < max; j++)
```

```
    sum += B[j];
```

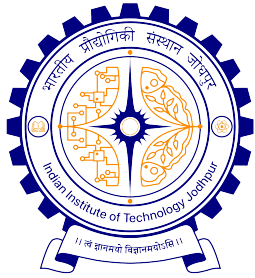


# Use

- An array element can be used wherever a simple variable (of data type float, char, int etc.) of the same type can be used.

Example:

```
scanf ("%d", &marks[i]);  
marks[i] = (int) (sqrt(21.089));  
A[3] = x + y;  
x = y - A[3];  
z = sin(A[i]) + cos(B[j]);
```

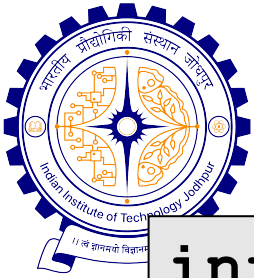


# Reading the elements of an array

- By reading them one element at a time

```
for (j=0; j<25; j++)  
    scanf ("%f", &a[j]);
```

- The ampersand (&) is necessary
- The elements can be entered all in one line or in different lines



# Reading into an array

```
int main()
{
    const int MAX_SIZE = 100;
    int i, size;
    float total, marks[MAX_SIZE];
    scanf("%d", &size);
    for (i=0, total=0; i<size; i++) {
        scanf("%f", &marks[i]);
        total = total + marks[i];
    }
    printf("Total = %f \n Avg = %f\n", total, total/size);
}
return 0;
```

## Output

4

2.5

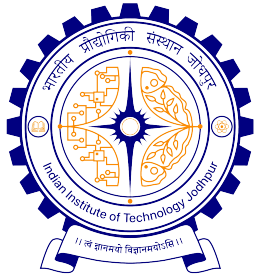
3.5

4.5

5

Total = 15.500000

Avg = 3.875000



# Printing the elements of an array

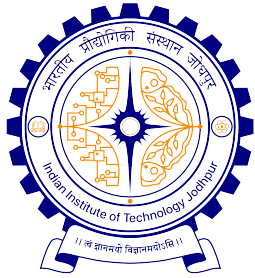
By printing them one element at a time

```
for (j=0; j<25; j++)  
    printf ("\n %f", a[j]);
```

The elements are printed one per line

```
printf ("\n");  
for (j=0; j<25; j++)  
    printf ("%f", a[j]);
```

The elements are printed all in one line (starting with a new line)

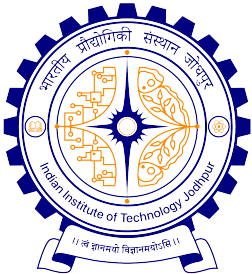


# Copy one array to the other

- By copying individual elements

```
for (j=0; j<25; j++)  
    a[j] = b[j];
```

- The element assignments will follow the rules of assignment expressions
- Destination array must have sufficient size



# Index Rule

- An array index must evaluate to an int between 0 and  $n-1$  where  $n$  is the number of elements in the array.
  - `marks[76]`
  - `marks[i*2+k]` // provided  $i*2+k$  is between 0 and 99

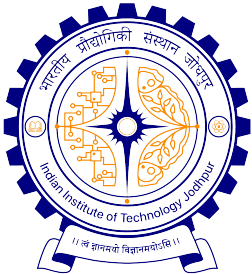
## In C, Array bounds are NOT checked



```
#define S 100  
marks[S] = 10;
```



```
if (0<=i && i<100)  
    marks[i] = 10;  
else printf ("Array index %d  
            out of range", i);
```



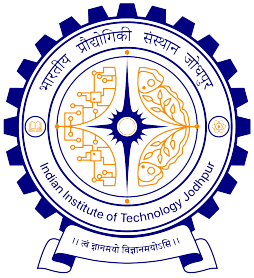
# Warning

**In C, Array bounds are NOT checked**

## Example

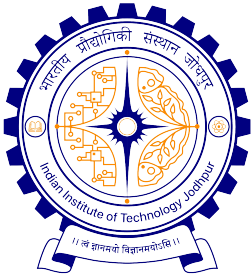
```
int    marks[5];  
:  
:  
marks[8] = 75;
```

- The above assignment would not necessarily cause an error
- It is the programmer's responsibility to avoid indexing off the end of an array
  - *Likely* to corrupt data
  - May cause a *segmentation fault*
  - Could expose system to a *security hole*!



# Warning

- You **can not**
  - use `=` to assign one array variable to another
  - use `==` to directly compare array variables
  - directly `scanf` or `printf` arrays
- But you **can** do these things on **array elements**.
- You **can** write functions to do them.



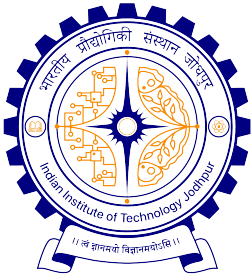
# Example

- Find the minimum of a set of 10 integers

```
int main()
{
    int  a[10], i, min;

    for  (i=0; i<10; i++)
        scanf ("%d", &a[i]);

    min = a[0];
    for  (i=1; i<10; i++){
        if  (a[i] < min)
            min = a[i];
    }
    printf ("\n Minimum is %d", min);
    return 0;
}
```



# Example

- Find the minimum of a set of 10 integers

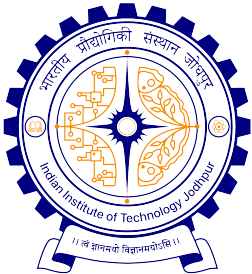
```
#define size 10

int main()
{
    int a[size], i, min;

    for (i=0; i<size; i++)
        scanf ("%d", &a[i]);

    min = a[0];
    for (i=1; i<size; i++){
        if (a[i] < min)
            min = a[i];
    }
    printf ("\n Minimum is %d", min);
    return 0;
}
```

Change only one  
line to change the  
problem size

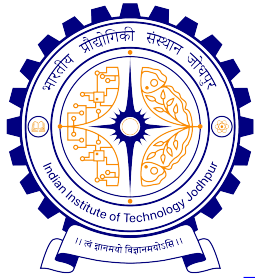


# Example: Computing CGPA

```
const int nsub = 6;
int main()
{
    int  grade_pt[nsub], cred[nsub], i,
        gp_sum=0, cred_sum=0;
    double gpa;

    for (i=0; i<nsub; i++)
        scanf ("%d %d", &grade_pt[i], &cred[i]);

    for (i=0; i<nsub; i++){
        gp_sum += grade_pt[i] * cred[i];
        cred_sum += cred[i];
    }
    gpa = ((float) gp_sum) / cred_sum;
    printf ("\n Grade point average:  is %.2lf", gpa);
    return 0;
}
```



# Character Arrays and Strings

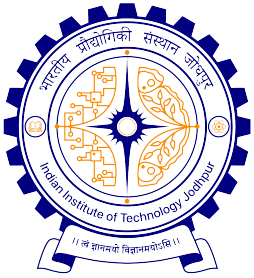
```
char C[8] = { 'a', 'b', 'h', 'i', 'j', 'i', 't', '\0' };
```

C[0] gets the value 'a', C[1] the value 'b', and so on. The last (7th) location receives the null character '\0'

- Null-terminated (last character is '\0') character arrays are also called strings
- Strings can be initialized in an alternative way. The last declaration is equivalent to:

```
char C[8] = "abhijit";
```

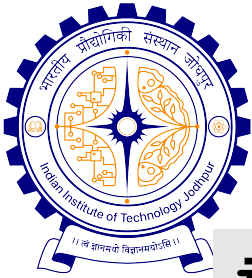
- The trailing null character is missing here. C automatically puts it at the end if you define it like this
- Note also that for individual characters, C uses single quotes, whereas for strings, it uses double quotes



# Reading strings: %s format

```
int main()  
{  
    char name[25];  
    scanf("%s", name);  
    printf("Name = %s \n", name);  
    return 0;  
}
```

**%s reads a string into a character array  
given the array name or start address.  
It ends the string with '\0'**



# An Example

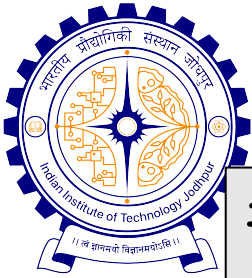
```
int main()  
{  
    #define SIZE 25  
    int i, count=0;  
    char name[SIZE];  
    scanf("%s", name);  
    printf("Name = %s \n", name);  
    for (i=0; name[i]!='\0'; i++)  
        if (name[i] == 'a') count++;  
    printf("Total a's = %d\n", count);  
    return 0;  
}
```

Satyanarayana

Name = Satyanarayana

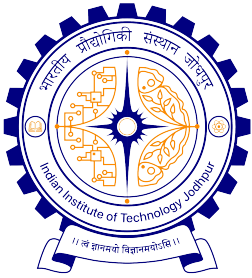
Total a's = 6

**Note that character strings read  
in %s format end with '\0'**



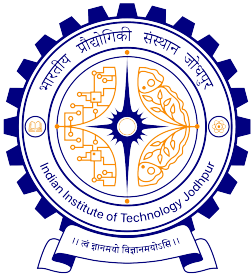
# Palindrome Checking

```
int main()
{
    const int SIZE = 25;
    int i, flag, count=0;
    char name[SIZE];
    scanf("%s", name);      /* Read Name */
    for (i=0; name[i]!='\0'; i++);    /* Find Length of String */
    printf("Total length = %d\n",i);
    count=i;  flag = 0;
    /* Loop below checks for palindrome by comparison*/
    for(i=0; i<count; i++) if (name[i]!=name[count-i-1]) flag = 1;
    if (flag ==0) printf ("%s is a Palindrome\n", name);
    else printf("%s is NOT a Palindrome\n", name);
    return 0;
}
```



# Binary Search

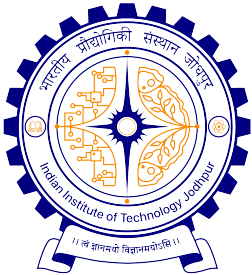
- Searching for an element  $k$  in a sorted array  $A$  with  $n$  elements
- Idea:
  - Choose the middle element  $A[n/2]$
  - If  $k == A[n/2]$ , we are done
  - If  $k < A[n/2]$ , search for  $k$  between  $A[0]$  and  $A[n/2 - 1]$
  - If  $k > A[n/2]$ , search for  $k$  between  $A[n/2 + 1]$  and  $A[n-1]$
  - Repeat until either  $k$  is found, or no more elements to search
- Requires less number of comparisons than linear search in the worst case ( $\log_2 n$  instead of  $n$ )



# Binary Search

```
int main() {
    int A[100], n, k, i, mid, low, high;
    scanf("%d %d", &n, &k);
    for (i=0; i<n; ++i) scanf("%d", &A[i]);
    low = 0; high = n - 1; mid = low + (high - low)/2;

    while (high >= low) {
        printf("low = %d, high = %d, mid = %d, A[%d] = %d\n",
low, high, mid, mid, A[mid]);
        if (A[mid] == k) {
            printf("%d is found\n", k);
            break;
        }
        if (k < A[mid]) high = mid - 1;
        else low = mid + 1;
        mid = low + (high - low)/2;
    }
    if (high < low) printf("%d is not found\n", k);
    return 0;
}
```



# Binary Search

## Output

8 21

9 11 14 17 19 20 23 27

low = 0, high = 7, mid = 3, A[3] = 17

low = 4, high = 7, mid = 5, A[5] = 20

low = 6, high = 7, mid = 6, A[6] = 23

21 is not found

8 14

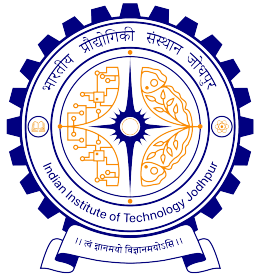
9 11 14 17 19 20 23 27

low = 0, high = 7, mid = 3, A[3] = 17

low = 0, high = 2, mid = 1, A[1] = 11

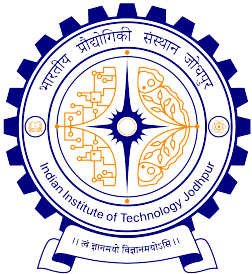
low = 2, high = 2, mid = 2, A[2] = 14

14 is found



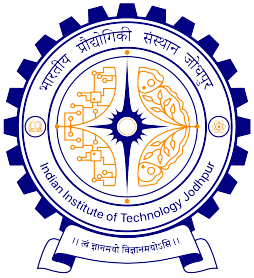
# Selection Sort

- Sort the elements of an array  $A$  with  $n$  elements in ascending order
- Basic Idea:
  - Find the min of the  $n$  elements, swap it with  $A[0]$  (so min is at  $A[0]$  now)
  - Now find the min of the remaining  $n-1$  elements, swap it with  $A[1]$  (so 2<sup>nd</sup> min is at  $A[1]$  now)
  - Continue until no more elements left



# Selection Sort

```
int main() {
    int A[100], n, i, j, k, min, pos, temp;
    scanf("%d", &n);
    for (i=0; i<n; ++i) scanf("%d", &A[i]);
    for (i = 0; i < n - 1; ++i) {
        min = A[i]; pos = i;
        for (j = i + 1; j < n; ++j) {
            if (A[j] < min) {
                min = A[j];
                pos = j;
            }
        }
        temp = A[i];
        A[i] = A[pos];
        A[pos] = temp;
        for (k=0; k<n; ++k) printf("%d  ", A[k]);
        printf("\n");
    }
    return 0;
}
```



# Selection Sort

## Output

6

7 12 5 15 17 9

5 12 7 15 17 9

5 7 12 15 17 9

5 7 9 15 17 12

5 7 9 12 17 15

5 7 9 12 15 17

8

9 8 7 6 5 4 3 2

2 8 7 6 5 4 3 9

2 3 7 6 5 4 8 9

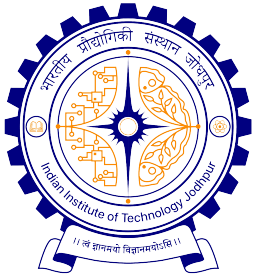
2 3 4 6 5 7 8 9

2 3 4 5 6 7 8 9

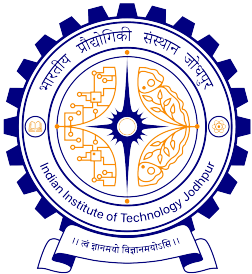
2 3 4 5 6 7 8 9

2 3 4 5 6 7 8 9

2 3 4 5 6 7 8 9

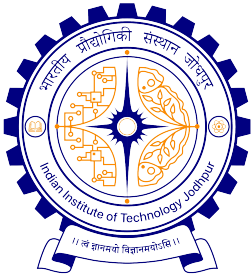


# String Handling



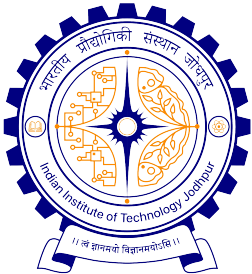
# Strong Library <string.h>

```
int main()
{
    char b[] = "Welcome IITJ Students";
    char a[50], c[50];
    int count = 0, i = 0;
    count = strlen(b);
    printf("The length is : %d\n", count);
    for (i = 0; i < count; ++i){
        a[i] = b[i];
    }
    a[i] = '\0';
    strcpy(c, b); // (des, src);
    if(strcmp(a, c) == 0)
        printf("Both same\n");
    return 0;
}
```



# Comparing two Strings

```
int main()
{
char s1[] = "This is IIT J.";
char s2[] = "This ia IIT J.";
if(strcmp(s1, s2) == 0)
    printf("Equal\n");
else if(strcmp(s1, s2) < 0)
    printf("s1 < s2\n");
else if(strcmp(s1, s2) > 0)
    printf("s1 > s2\n");
}
```



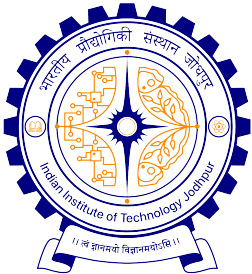
# Copying Strings

```
int main()
{
char src [11] = "1Abcde7890";
char dest1[50], dest2[5], dest3[50], dest4[50], dest5;
strcpy(dest1, src);
printf("dest1: %s\n", dest1);
strcpy(dest2, src); // working but will avoid for other compilers
printf("dest2: %s\n", dest2);
strncpy(dest3, src, 5);
dest3[5] = '\\0'; /// might not need but it is safer to ensure with \\0
printf("dest3: %s\n", dest3);
strncpy(dest4, src, 40); // will print up to null
printf("dest4: %s\n", dest4);
}
```



# Reading string using fgets

```
#include <stdio.h>
#include <string.h>
#define BUFFER_SIZE 100
int main() {
    char buffer[BUFFER_SIZE];
    printf("Enter a string: "); // Read input from the standard input stream (stdin)
    if (fgets(buffer, sizeof(buffer), stdin) != NULL)
        size_t length = strlen(buffer);
        if (length > 0 && buffer[length - 1] == '\\n') {
            buffer[length - 1] = '\\0'; }
        printf("You entered: %s\\n", buffer);
    } else {
        printf("Error reading input.\\n");
    }
    return 0;
}
```



# fgets function

```
char *fgets(char *str, int n, FILE *stream);
```

A pointer to the character array (buffer) where the read string will be stored.

The maximum number of characters to read, including the null terminator (`\0`). fgets will read at most `n-1` characters.

A pointer to the input stream. Use `stdin` for reading from the keyboard or a file pointer (e.g., from `fopen()`) for reading from a file.

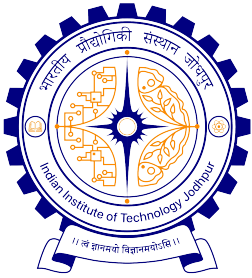
- `fgets()` reads until a **newline character (`\n`)**, the **end of the file (EOF)**, or **`n-1` characters** have been read, whichever comes first.
- It automatically **appends a null character (`\0`)** at the end of the string in the buffer.
- It returns **`str` on success** and **`NULL` if an error** occurs or the end-of-file is reached with no characters read.

the user presses 'enter' after typing his name and fgets also stores "\n" which hinders with strlen/strcat etc to get rid of it  
we are using the strchr step in this code!!



# String Concatenation

```
#define BUFFER_SIZE 100
int main() {
    char destination[BUFFER_SIZE] = "Hello, ";
    char source[BUFFER_SIZE / 2]; // Ensure source is smaller than remaining dest space.
    Get user input safely using fgets
    printf("Enter a name to greet: ");
    if (fgets(source, sizeof(source), stdin) != NULL) { // Use strchr to find and
        remove the trailing newline character // strchr finds the length of the initial
        segment of source that consists entirely of characters NOT in "\n". This gives the
        index of the newline character.
        size_t newline_pos = strchr(source, "\n"); printf("%zu\n", newline_pos);
        if (newline_pos < sizeof(source)) {
            source[newline_pos] = '\0';
        } // Use strcat to append the modified source string to the destination. The
        destination buffer must have enough space for the combined strings and the null
        terminator.
        strcat(destination, source); //Print the final concatenated string
        printf("Concatenated string: %s\n", destination);
    }
    else printf("Error reading input.\n");
    return 0;
}
```



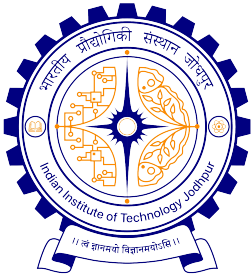
# About `strcspn(s1,s2)` and `strcat(s1,s2)`

- **`strcspn(source, "\n")`:**

- This function calculates the length of the initial segment of the source string that consists entirely of characters *not* in the second string argument (which is just `"\n"` in this case).
- The return value is the index of the first occurrence of `\n` in source.
- `source[newline_pos] = '\0'`; replaces that newline character with a null terminator `\0`, effectively removing the newline from the string and ensuring proper string termination.

- **`strcat(destination, source)`:**

- This function appends a copy of the source string to the end of the destination string.
- It overwrites the null terminator originally at the end of the destination string with the first character of the source string, and then adds a new null terminator at the end of the combined result.
- It is crucial to ensure that the destination array is large enough to hold the final combined string to avoid a buffer overflow.



- Slide Courtesy: *The C Programming Language*, 2<sup>nd</sup> edition, by Kernighan and Ritchie and from *C: How to Program*, 5<sup>th</sup> and 6<sup>th</sup> editions, by Deitel and Deitel)